

Supplementary Material: Oblivious Multi-Way Band Joins

This supplementary material expands the main paper section by section, in the same order. Each section states what it covers.

A EXTENDED BACKGROUND MATERIAL

A.1 Detailed Oblivious Primitives

Oblivious algorithms rely on data-independent primitives that operate on tables (arrays of rows) where each row contains values for multiple columns. These primitives ensure access patterns reveal no information about the input data.

Oblivious Sorting. Oblivious Sorting applies to a table using a sorting order function that takes two rows and returns -1 (first row is “smaller”), 1 (second row is “smaller”), or 0 when the two compare equal, in which case the precedence ordering of the entry types settles the tie. Following the shuffle-then-sort paradigm of Hamada et al. [4], the algorithm first obviously shuffles the data to randomize positions, then performs an ordinary comparison sort: the comparison outcomes are those of a randomly permuted sequence, so they carry no information about the original data distribution. This ensures that regardless of the input values, the memory access patterns reveal nothing about the actual data. Our implementation achieves $O(n \log n)$ complexity this way [4]; deterministic sorting networks are the alternative, at $O(n \log^2 n)$ for Batchier’s [1] and $O(n \log n)$ for zig-zag sort [2] at a large constant factor.

Oblivious Distribution. Oblivious Distribution moves rows to computed target positions within a table without revealing information about where rows are being moved or how many rows end up in each location. This primitive is essential in the ODBJ framework for repositioning rows according to their multiplicities before expansion. The algorithm uses a series of oblivious sorting and permutation operations to achieve the desired redistribution while ensuring that the access pattern depends only on the table size and target position computation, not on the actual row values or movement patterns.

Oblivious Expansion. Oblivious Expansion creates multiple copies of rows based on precomputed multiplicities, ensuring that the duplication process reveals no information about how many copies each row requires. This primitive works in conjunction with oblivious distribution to construct join result tables where each row appears exactly as many times as required by the join semantics. The challenge lies in handling variable expansion factors obviously—some rows may need many copies while others need few, but the algorithm must access memory in a pattern that depends only on the maximum possible expansion factor, not the actual requirements.

Map. Map applies an oblivious function to every row of a table. The function operates independently on each row, maintaining data-independent access patterns by reading and writing at predetermined positions within each row. This primitive enables row transformations while preserving obliviousness, such as computing new attributes, applying selection conditions, or reformatting

tuple structures. The resulting table may contain modified rows but maintains the same size as the input table.

LinearPass. LinearPass takes a table and an oblivious function that operates on a fixed number of rows (the window). The function reads from and writes to fixed locations within the window. Linear pass places this fixed-size sliding window on the table and applies the function to each window position as it moves through the table. This maintains data-independent access patterns since the window size and movement are predetermined, ensuring that the memory access sequence depends only on the table size and window size, not on the actual data values encountered.

ParallelPass. ParallelPass takes a table and an oblivious function that operates on two rows (one from each table). The function reads from and writes to fixed locations within these two rows. Parallel pass processes corresponding row pairs with equal indexes from two tables of the same size, applying the oblivious function to each pair sequentially. This maintains data-independent access patterns since the function operates on predetermined fixed locations, ensuring that the memory access sequence depends only on the table sizes and function structure, not on the actual data values.

A.2 Generalized Hypertree Decomposition

Generalized Hypertree Decomposition (GHD) [3] provides a systematic way to transform cyclic queries into acyclic ones. The key idea is to group relations into “bags” arranged in a tree structure, where each bag may contain multiple relations. By pre-computing joins within each bag, we create an acyclic structure that can be processed with tree-based algorithms.

The efficiency of this transformation depends on the Generalized Hypertree Width (GHW) of the query—the minimum number of relations needed in any bag across all possible decompositions. Acyclic queries naturally have $\text{GHW} = 1$ (no grouping needed), while the triangle query has $\text{GHW} = 2$, and an ℓ -cycle has $\text{GHW} = \lceil \ell/2 \rceil$.

The transformation can increase data size exponentially: from N to potentially N^{GHW} , as bags may contain Cartesian products. This exponential blowup makes GHD transformation impractical for queries with large GHW, especially in the oblivious setting where we cannot optimize based on actual data distributions.

A.3 Yannakakis Algorithm Details

The Yannakakis algorithm [7], developed by Mihalis Yannakakis in 1981, provides an elegant solution for evaluating acyclic multi-way joins with optimal complexity. The algorithm achieves $O(N + \text{OUT})$ time, where N is the total input size and OUT is the output size—this is optimal because any algorithm must at least read the input and write the output.

The key insight is to exploit the tree structure of acyclic queries through a two-phase approach, avoiding the exponential blowup of naive join ordering.

The Two-Phase Approach. In the bottom-up phase, the algorithm performs semi-join reductions starting from the leaves of the join tree. Each child table sends information to its parent about which values actually exist, allowing the parent to eliminate tuples that have no matching partners. This process continues up to the root, with each table keeping only tuples that can contribute to the final result based on their subtree.

The top-down phase then propagates constraints from the root back to the leaves. Starting from the filtered root table containing only tuples that exist in the final result, each parent informs its children about which values remain valid in the global context. This ensures that every tuple in the final result participates in the complete join across all tables.

While Yannakakis’ algorithm is optimal for non-oblivious settings, it reveals information through its access patterns—the size of intermediate results reveals the selectivity of different join conditions. Our work extends this algorithm to maintain its efficiency while hiding these access patterns.

B DETAILED ALGORITHM SPECIFICATIONS

This section gives the complete pseudocode for every helper algorithm the main paper’s Algorithm Detail section invokes by name.

B.1 Utility Functions

These core primitives are used throughout the algorithm to process tables obliviously.

Algorithm 1 LinearPass: Apply window function across table with sliding window size 2

```

1: function LINEARPASS( $R$ ,  $WindowFunc$ )
2:   for  $i = 1$  to  $|R| - 1$  do
3:      $window \leftarrow R[i : i + 1]$   $\triangleright$  Extract window of size 2
4:     WINDOWFUNC( $window$ ,  $i$ )  $\triangleright$  Apply function to window
5:   end for
6:   return  $R$   $\triangleright$  Return modified table
7: end function
```

Algorithm 2 Map: Apply transformation function to each row independently

```

1: function MAP( $R$ ,  $TransformFunc$ )
2:    $R_{out} \leftarrow []$   $\triangleright$  Initialize output table
3:   for  $i = 1$  to  $|R|$  do
4:      $R_{out}[i] \leftarrow TRANSFORMFUNC(R[i], i)$ 
5:   end for
6:   return  $R_{out}$ 
7: end function
```

Algorithm 3 ParallelPass: Apply window function to aligned pairs from two tables

```

1: function PARALLELPASS( $R_1$ ,  $R_2$ ,  $WindowFunc$ )
Require:  $|R_1| = |R_2|$   $\triangleright$  Tables must have same size
2:   for  $i = 1$  to  $|R_1|$  do
3:      $window \leftarrow [R_1[i], R_2[i]]$   $\triangleright$  Create window from aligned pair
4:     WINDOWFUNC( $window$ ,  $i$ )  $\triangleright$  Apply function to modify in-place
5:   end for
6:   return ( $R_1$ ,  $R_2$ )  $\triangleright$  Return modified tables
7: end function
```

B.2 Initialization Functions

Algorithm 4 Initialize augmented tables: Add metadata columns

```

1: function INITIALIZEALLTABLES( $T$ )
2:   for all nodes  $v \in V$  do
3:      $R_v \leftarrow MAP(R_v, AddMetadataColumns)$ 
4:     LINEARPASS( $R_v$ , WindowSetOriginalIndex)
5:   end for
6:   return  $T$ 
7: end function
```

Algorithm 5 Add Metadata Columns: Map function to extend tuples with null metadata

```

1: function ADDMETADATACOLUMNS( $t$ ,  $index$ )
2:    $t.orig\_idx \leftarrow 0$ 
3:    $t.\alpha_{local} \leftarrow null$ 
4:    $t.\alpha_{final} \leftarrow null$ 
5:    $t.F_{sum} \leftarrow null$ 
6:   return  $t$ 
7: end function
```

Algorithm 6 Window Set Original Index: Assign sequential indices

```

1: function WINDOWSETORIGINALINDEX( $window$ )
2:    $window[1].orig\_idx \leftarrow window[0].orig\_idx + 1$ 
3: end function
```

B.3 Bottom-Up Phase Algorithms

Algorithm 7 Bottom-Up Phase: Compute local multiplicities from leaves to root

```

1: function BOTTOMUPPHASE( $T, root$ )
2:    $order \leftarrow \text{POSTORDERTRAVERSAL}(T, root)$ 
3:   for all nodes  $v$  in  $order$  do
4:     if  $v$  is a leaf then
5:       for all tuple  $t \in R_v$  do
6:          $t.\alpha_{\text{local}} \leftarrow 1$ 
7:       end for
8:     else
9:       for all child nodes  $c$  of  $v$  do
10:         $R_v \leftarrow \text{COMPUTELOCALMULTIPLICITIES}(R_v, R_c,$ 
11:           $C(v, c))$ 
12:      end for
13:    end if
14:  return  $T$ 
15: end function

```

Algorithm 8 Post-Order Traversal

```

1: function POSTORDERTRAVERSAL( $T, root$ )
2:    $order \leftarrow$  empty list
3:   for all child nodes  $c$  of  $root$  do
4:      $order \leftarrow order + \text{POSTORDERTRAVERSAL}(T, c)$ 
5:   end for
6:   Append  $root$  to  $order$ 
7:   return  $order$ 
8: end function

```

Algorithm 9 Compute Local Multiplicities

```

1: function COMPUTELOCALMULTIPLICITIES( $R_{\text{target}}, R_{\text{source}}, \theta$ )
2:    $R_{\text{comb}} \leftarrow \text{COMBINE\_TABLE}(R_{\text{target}}, R_{\text{source}}, \theta)$ 
3:    $R_{\text{comb}} \leftarrow \text{MAP}(R_{\text{comb}}, \lambda e : (e.\text{local\_sum} \leftarrow$ 
4:      $e.\alpha_{\text{local}}, e.\text{local\_interval} \leftarrow 0, e))$ 
5:   OBLIVIOUSSORT( $R_{\text{comb}}, \text{ComparatorJoinAttr}$ )
6:   LINEARPASS( $R_{\text{comb}}, \text{WindowComputeLocalSum}$ )
7:   OBLIVIOUSSORT( $R_{\text{comb}}, \text{ComparatorPairwise}$ )
8:   LINEARPASS( $R_{\text{comb}}, \text{WindowComputeLocalInterval}$ )
9:   OBLIVIOUSSORT( $R_{\text{comb}}, \text{ComparatorEndFirst}$ )
10:   $R_{\text{truncated}} \leftarrow R_{\text{comb}}[1 : |R_{\text{target}}|]$ 
11:  PARALLELPASS( $R_{\text{truncated}}, R_{\text{target}}, \text{UpdateTargetMultiplicity}$ )
12: return  $R_{\text{target}}$ 
13: end function

```

Algorithm 10 Combine Table: Create boundary entries

```

1: function COMBINE\_TABLE( $R_{\text{target}}, R_{\text{source}}, \theta$ )
2:    $((d_1, eq_1), (d_2, eq_2)) \leftarrow \theta$ 
3:    $R'_{\text{source}} \leftarrow \text{MAP}(R_{\text{source}}, \text{create SOURCE entries})$ 
4:    $R'_{\text{begin}} \leftarrow \text{MAP}(R_{\text{target}}, \text{create START entries at join\_attr} +$ 
5:      $d_1)$ 
6:    $R'_{\text{end}} \leftarrow \text{MAP}(R_{\text{target}}, \text{create END entries at join\_attr} + d_2)$ 
7:    $R_{\text{comb}} \leftarrow R'_{\text{source}} + R'_{\text{begin}} + R'_{\text{end}}$ 
8:   return  $R_{\text{comb}}$ 
9: end function

```

B.4 Top-Down Phase Algorithms

Algorithm 11 Top-Down Phase: Propagate final multiplicities

```

1: function TOPDOWNPHASE( $T, root$ )
2:   for all tuple  $t \in R_{\text{root}}$  do
3:      $t.\alpha_{\text{final}} \leftarrow t.\alpha_{\text{local}}$ 
4:   end for
5:   for all nodes  $v$  in pre-order traversal do
6:     for all child nodes  $c$  of  $v$  do
7:        $R_c \leftarrow \text{PROPAGATEFINALMULTIPLICITIES}(R_v, R_c,$ 
8:          $C(v, c))$ 
9:     end for
10:  return  $T$ 
11: end function

```

Algorithm 12 Propagate Final Multiplicities

```

1: function PROPAGATEFINALMULTIPLICITIES( $R_{\text{source}}, R_{\text{target}}, \theta$ )
2:    $\theta' \leftarrow ((-d_2, eq_2), (-d_1, eq_1))$   $\triangleright$  roles reversed: invert
3:    $R_{\text{comb}} \leftarrow \text{COMBINE\_TABLE}(R_{\text{target}}, R_{\text{source}}, \theta')$ 
4:   Initialize  $w_{\text{local}}, C_{\text{foreign}}, \text{foreign\_interval}$  fields
5:   OBLIVIOUSSORT( $R_{\text{comb}}, \text{ComparatorJoinAttr}$ )
6:   LINEARPASS( $R_{\text{comb}}, \text{WindowComputeForeignSum}$ )
7:   OBLIVIOUSSORT( $R_{\text{comb}}, \text{ComparatorPairwise}$ )
8:   LINEARPASS( $R_{\text{comb}}, \text{WindowComputeForeignInterval}$ )
9:   OBLIVIOUSSORT( $R_{\text{comb}}, \text{ComparatorEndFirst}$ )
10:   $R_{\text{truncated}} \leftarrow R_{\text{comb}}[1 : |R_{\text{target}}|]$ 
11:  PARALLELPASS( $R_{\text{truncated}}, R_{\text{target}}, \text{UpdateTargetFinalMulti}$ 
12:     $\text{plicity})$ 
13: return  $R_{\text{target}}$ 
14: end function

```

B.5 Comparator Functions

Algorithm 13 Comparator Join Attribute

```

1: function COMPARATORJOINATTR( $e_1, e_2$ )
2:   if  $e_1.join\_attr \neq e_2.join\_attr$  then return
    $sign(e_1.join\_attr - e_2.join\_attr)$ 
3:   else
4:      $p_1 \leftarrow GETPRECEDENCE((e_1.type, e_1.equality))$ 
5:      $p_2 \leftarrow GETPRECEDENCE((e_2.type, e_2.equality))$  return
    $sign(p_1 - p_2)$ 
6:   end if
7: end function

```

Algorithm 14 Comparator Pairwise

```

1: function COMPARATORPAIRWISE( $e_1, e_2$ )
2:   if  $e_1.type \in \{START, END\}$  and  $e_2.type = SOURCE$  then
   return -1
3:   else if  $e_1.type = SOURCE$  and  $e_2.type \in \{START, END\}$ 
   then return 1
4:   else if  $e_1.orig\_idx \neq e_2.orig\_idx$  then return
    $sign(e_1.orig\_idx - e_2.orig\_idx)$ 
5:   else if  $e_1.type = START$  and  $e_2.type = END$  then return
   -1
6:   else if  $e_1.type = END$  and  $e_2.type = START$  then return
   1
7:   elsereturn 0
8:   end if
9: end function

```

Algorithm 15 Comparator End First

```

1: function COMPARATORENDFIRST( $e_1, e_2$ )
2:   if  $e_1.type = END$  and  $e_2.type \neq END$  then return -1
3:   else if  $e_1.type \neq END$  and  $e_2.type = END$  then return 1
4:   elsereturn  $sign(e_1.orig\_idx - e_2.orig\_idx)$ 
5:   end if
6: end function

```

Algorithm 16 Alignment Key Comparator

```

1: function ALIGNMENTKEYCOMPARATOR( $t_1, t_2$ )
2:   if  $t_1.alignment\_key \neq t_2.alignment\_key$  then return
    $sign(t_1.alignment\_key - t_2.alignment\_key)$ 
3:   else if  $t_1.join\_attr \neq t_2.join\_attr$  then return
    $sign(t_1.join\_attr - t_2.join\_attr)$ 
4:   elsereturn  $sign(t_1.copy\_index - t_2.copy\_index)$ 
5:   end if
6: end function

```

B.6 Window Functions

Algorithm 17 Window Compute Local Sum

```

1: function WINDOWCOMPUTELOCALSUM( $window$ )
2:   if  $window[1].type = SOURCE$  then
3:      $window[1].local\_sum \leftarrow window[0].local\_sum +$ 
    $window[1].\alpha_{local}$ 
4:   else
5:      $window[1].local\_sum \leftarrow window[0].local\_sum$ 
6:   end if
7: end function

```

Algorithm 18 Window Compute Local Interval

```

1: function WINDOWCOMPUTELOCALINTERVAL( $window$ )
2:   if  $window[0].type = START$  and  $window[1].type = END$ 
   then
3:      $window[1].local\_interval \leftarrow window[1].local\_sum -$ 
    $window[0].local\_sum$ 
4:   end if
5: end function

```

Algorithm 19 Window Compute Foreign Sum

```

1: function WINDOWCOMPUTEFOREIGNSUM( $window$ )
2:   if  $window[1].type = START$  then
3:      $window[1].w_{local} \leftarrow window[0].w_{local} +$ 
    $window[1].\alpha_{local}$ 
4:      $window[1].C_{foreign} \leftarrow window[0].C_{foreign}$ 
5:   else if  $window[1].type = END$  then
6:      $window[1].w_{local} \leftarrow window[0].w_{local} -$ 
    $window[1].\alpha_{local}$ 
7:      $window[1].C_{foreign} \leftarrow window[0].C_{foreign}$ 
8:   else if  $window[1].type = SOURCE$  then
9:      $window[1].w_{local} \leftarrow window[0].w_{local}$ 
10:     $window[1].C_{foreign} \leftarrow window[0].C_{foreign} +$ 
    $window[1].\alpha_{final}/window[1].w_{local}$ 
11:   end if
12: end function

```

Algorithm 20 Window Compute Foreign Interval

```

1: function WINDOWCOMPUTEFOREIGNINTERVAL( $window$ )
2:   if  $window[0].type = START$  and  $window[1].type = END$ 
   then
3:      $window[1].foreign\_interval \leftarrow window[1].C_{foreign} -$ 
    $window[0].C_{foreign}$ 
4:      $window[1].F_{sum} \leftarrow window[0].C_{foreign}$ 
5:   end if
6: end function

```

B.7 Update and Helper Functions

Algorithm 21 Update Target Multiplicity

```

1: function UPDATETARGETMULTIPLICITY( $e_{combined}, e_{target}$ )
2:    $e_{target}.\alpha_{local} \leftarrow e_{target}.\alpha_{local} \times e_{combined}.local\_interval$ 
3: end function

```

Algorithm 22 Update Target Final Multiplicity

```

1: function UPDATETARGETFINALMULTIPLICITY( $e, t$ )
2:    $t.\alpha_{final} \leftarrow e.foreign\_interval \times t.\alpha_{local}$ 
3:    $t.F_{sum} \leftarrow e.F_{sum}$ 
4: end function

```

Algorithm 23 Get Entry Type Precedence

```

1: function GETPRECEDENCE( $(entry\_type, equality\_type)$ )
2:   if  $(entry\_type, equality\_type) \in \{(START, EQ), (END, NEQ)\}$  then return 1
3:   else if  $(entry\_type, equality\_type) = (SOURCE, null)$  then return 2
4:   elsereturn 3
5:   end if
6: end function

```

B.8 Result Construction Algorithms

Algorithm 24 Result Construction

```

1: function CONSTRUCTJOINRESULT( $T, root$ )
2:    $result \leftarrow OBLIVIOUSEXPAND(R_{root})$ 
3:   for all nodes  $v$  in pre-order traversal do
4:     for all child nodes  $c$  of  $v$  do
5:        $R_c^{expanded} \leftarrow OBLIVIOUSEXPAND(R_c)$ 
6:        $result \leftarrow ALIGNANDCONCATENATE(result, R_c^{expanded})$ 
7:     end for
8:   end forreturn  $result$ 
9: end function

```

Algorithm 25 Align and Concatenate

```

1: function ALIGNANDCONCATENATE( $R_{accumulator}, R_{child}$ )
2:    $OBLIVIOUSSORT(R_{accumulator}, JoinThenOtherAttributes)$ 
3:    $LINEARPASS(R_{child}, SetCopyIndex)$ 
4:    $LINEARPASS(R_{child}, ComputeAlignmentKey)$ 
5:    $OBLIVIOUSSORT(R_{child}, AlignmentKeyComparator)$ 
6:   return  $HORIZONTALCONCATENATE(R_{accumulator}, R_{child})$ 
7: end function

```

Algorithm 26 Set Copy Index

```

1: function SETCOPYINDEX( $window$ )
2:    $same \leftarrow (window[0].orig\_idx == window[1].orig\_idx) \in \{0, 1\}$ 
3:    $window[1].copy\_index \leftarrow same \cdot (window[0].copy\_index + 1)$ 
4: end function

```

Algorithm 27 Compute Alignment Key

```

1: function COMPUTEALIGNMENTKEY( $tuple$ )
2:    $tuple.alignment\_key \leftarrow tuple.F_{sum} + \lfloor tuple.copy\_index / tuple.\alpha_{local} \rfloor$ 
3: end function

```

C WORKED EXAMPLES

The dual-entry sweep and the multiplicity bookkeeping are easier to follow on a concrete instance than in the formal definitions of the main paper. This section traces two small queries end to end. Both were run through the released implementation with the intermediate arrays dumped at each step, so every number below is the engine's own output rather than a hand computation. We write START, END and SOURCE for START, END and SOURCE entries.

C.1 A two-table band join

Take a parent table P with join values 10, 20, 30 (tuples p_0, p_1, p_2) and a child table C with join values 8, 10, 19, 20 (tuples $c_0 \dots c_3$), under the *asymmetric* band

$$c.a > p.a - 3 \quad \text{and} \quad c.a \leq p.a,$$

that is $c.a \in p.a + (-3, 0]$, so $d_1 = -3$ with an open boundary and $d_2 = 0$ with a closed one. An asymmetric band is the interesting case: for a symmetric window the top-down inversion of Section ?? of the main paper is invisible.

Bottom-up. Each parent contributes a START at $p.a - 3$ and an END at $p.a$; each child contributes a SOURCE at its own value. Sorting by join attribute (ties by the precedence of Table ?? of the main paper) and running the linear pass gives:

Pos	Type	Value	Tuple	Cumulative sum
0	START	7	p_0	1
1	SOURCE	8	c_0	2
2	SOURCE	10	c_1	3
3	END	10	p_0	3
4	START	17	p_1	3
5	SOURCE	19	c_2	4
6	SOURCE	20	c_3	5
7	END	20	p_1	5
8	START	27	p_2	5
9	END	30	p_2	5

The counter starts at 1 rather than 0 because entries are initialised with their own α_{local} ; the difference taken next cancels the offset. Pairing each parent's two markers and subtracting yields its number of matching children:

Parent	Sum at START	Sum at END	Interval
p_0	1	3	2
p_1	3	5	2
p_2	5	5	0

So p_0 matches $\{c_0, c_1\}$, p_1 matches $\{c_2, c_3\}$, and p_2 matches nothing: its window $(27, 30]$ contains no child value. These are exactly the parents' α_{local} values after the edge, and p_2 's zero correctly removes it from the result.

Top-down. The roles now reverse, and with them the constraint: the child carries the markers, so $c.a \in p.a + (-3, 0]$ is rewritten as $p.a \in c.a + [0, 3)$, each equality flag travelling with its own endpoint. Child c_0 (value 8) therefore spans $[8, 11)$, not $[5, 8)$. The sweep maintains the running weight w_{local} of child ranges covering the current position and adds $\alpha_{\text{final}}/w_{\text{local}}$ at each parent SOURCE:

Pos	Type	Value	Tuple	w_{local}	C_{foreign}
0	START	8	c_0	1	0
1	START	10	c_1	2	0
2	SOURCE	10	p_0	2	1
3	END	11	c_0	1	1
4	END	13	c_1	0	1
5	START	19	c_2	1	1
6	START	20	c_3	2	1
7	SOURCE	20	p_1	2	2
8	END	22	c_2	1	2
9	END	23	c_3	0	2
10	SOURCE	30	p_2	0	2

Reading the difference across each child's marker pair gives its foreign multiplicity, and the value at its START gives F_{sum} :

Child	Value	α_{local}	α_{final}	F_{sum}
c_0	8	1	1	0
c_1	10	1	1	0
c_2	19	1	1	1
c_3	20	1	1	1

Every child appears once, and F_{sum} orders them into the two output blocks: c_0 and c_1 at offset 0 (both join p_0), c_2 and c_3 at offset 1 (both join p_1). Expansion and alignment then emit the four result rows $(10, 8)$, $(10, 10)$, $(20, 19)$, $(20, 20)$, which is the join computed directly.

C.2 A four-table star

The second instance exercises the case that makes Phase 4 delicate: a *centre* node with a parent and two children. The tree is $\text{root} \rightarrow \text{centre} \rightarrow \{\text{leaf}_A, \text{leaf}_B\}$, joined on equality, with

root $(1, 5), (2, 7)$ keyed by R_{KEY}
 centre $(1, 5, 100), (2, 5, 200), (3, 7, 100)$ keyed by $M_{\text{RKEY}}, M_{\text{LKEY}}$
 leaf_A $(1, 100), (2, 200)$
 leaf_B $(1, 100), (2, 100), (3, 200)$

The bottom-up pass gives the centre tuples local multiplicities $1 \times 2 = 2$, $1 \times 1 = 1$ and $1 \times 2 = 2$ respectively — one matching leaf_A tuple times the number of matching leaf_B tuples — so the root

tuples inherit 3 and 2, and the output has 5 rows. The top-down pass then splits those totals back down, and the alignment keys interleave each centre tuple's copies against the two leaf tables so that the cross-product of the two sibling subtrees is enumerated once. The engine emits

root	centre	leaf_A	leaf_B
(1, 5)	(1, 5, 100)	(1, 100)	(1, 100)
(1, 5)	(1, 5, 100)	(1, 100)	(2, 100)
(2, 7)	(3, 7, 100)	(1, 100)	(1, 100)
(2, 7)	(3, 7, 100)	(1, 100)	(2, 100)
(1, 5)	(2, 5, 200)	(2, 200)	(3, 200)

which is the five-row join. Note that the two copies of centre tuple 1 differ only in their leaf_B column, and the two copies of centre tuple 3 likewise: this pairing is what the canonical sort of the accumulated result establishes at each edge, and folding the subtrees into local accumulators instead would mispair them.

D COMPLETE SECURITY PROOFS

This section proves the lemmas behind the four-level argument of the main paper's Security Analysis, level by level.

D.1 Base Component Security

We prove that our custom window functions, comparators, and update functions can be converted to oblivious implementations. Our conversion strategy relies on two key techniques:

- (1) **Arithmetic conversion:** Replace all conditional branches with arithmetic operations using 0/1 predicates. For any condition, we compute a predicate $p \in \{0, 1\}$ and use multiplication: $\text{result} = p \cdot \text{value}_{\text{true}} + (1 - p) \cdot \text{value}_{\text{false}}$.
- (2) **Access pattern uniformity:** Ensure all execution paths access the same memory locations in the same order, regardless of data values.

This approach transforms data-dependent control flow into data-oblivious arithmetic operations, ensuring that the memory access pattern is independent of the input data values.

D.1.1 Window Functions.

LEMMA 1. *WINDOWCOMPUTELOCALSUM (Algorithm 17) can be converted to an oblivious implementation.*

PROOF. The function's conditional logic on entry type can be converted to oblivious form:

- (1) **Access pattern:** Always reads $\text{window}[0].\text{local_sum}$ and $\text{window}[1].\text{type}$, $\text{window}[1].\alpha_{\text{local}}$, and always writes to $\text{window}[1].\text{local_sum}$.
- (2) **Arithmetic conversion:** The conditional branch becomes:

$$\begin{aligned} \text{is_source} &= (\text{window}[1].\text{type} == \text{SOURCE}) \in \{0, 1\} \\ \text{window}[1].\text{local_sum} &= \text{window}[0].\text{local_sum} \\ &\quad + \text{is_source} \cdot \text{window}[1].\alpha_{\text{local}} \end{aligned}$$

This eliminates the conditional branch while preserving functionality: when SOURCE, adds α_{local} ; otherwise adds 0. $\square$

LEMMA 2. *WINDOWCOMPUTELOCALINTERVAL (Algorithm 18) can be converted to an oblivious implementation.*

PROOF. The function’s conditional interval computation can be made oblivious:

- (1) **Access pattern:** Always read `window[0]` and `window[1]` fields, always write to `window[1].local_interval`.
- (2) **Arithmetic conversion:** The conditional check becomes:

$$\begin{aligned} \text{is_pair} &= (\text{window}[0].\text{type} == \text{START}) \\ &\quad \cdot (\text{window}[1].\text{type} == \text{END}) \in \{0, 1\} \\ \text{interval} &= \text{window}[1].\text{local_sum} \\ &\quad - \text{window}[0].\text{local_sum} \\ \text{window}[1].\text{local_interval} &= \text{is_pair} \cdot \text{interval} \\ &\quad + (1 - \text{is_pair}) \cdot \text{window}[1].\text{local_interval} \end{aligned}$$

The write always happens (either new interval or preserving existing value). $\square$

LEMMA 3. *WINDOWCOMPUTEFOREIGNSUM (Algorithm 19) can be converted to an oblivious implementation.*

PROOF. The function’s three-way branch can be converted to arithmetic operations:

- (1) **Access pattern:** Always read both window slots, always write w_{local} and C_{foreign} of `window[1]`.
- (2) **Arithmetic conversion:** The type-based branching becomes:

$$\begin{aligned} \text{is_start} &= (\text{window}[1].\text{type} == \text{START}) \in \{0, 1\} \\ \text{is_end} &= (\text{window}[1].\text{type} == \text{END}) \in \{0, 1\} \\ \text{is_source} &= (\text{window}[1].\text{type} == \text{SOURCE}) \in \{0, 1\} \\ \text{weight_delta} &= \text{is_start} \cdot \text{window}[1].\alpha_{\text{local}} \\ &\quad - \text{is_end} \cdot \text{window}[1].\alpha_{\text{local}} \\ \text{window}[1].w_{\text{local}} &= \text{window}[0].w_{\text{local}} + \text{weight_delta} \\ \text{is_zero} &= (\text{window}[0].w_{\text{local}} == 0) \in \{0, 1\} \\ \text{safe_denom} &= \text{window}[0].w_{\text{local}} + \text{is_zero} \\ \text{foreign_delta} &= \text{is_source} \cdot (\text{window}[1].\alpha_{\text{final}} / \text{safe_denom}) \\ \text{window}[1].C_{\text{foreign}} &= \text{window}[0].C_{\text{foreign}} + \text{foreign_delta} \end{aligned}$$

The safe denominator ensures division is never by zero: it is the weight itself wherever that is non-zero, and 1 where it is not. The substitution never perturbs a retained value, since the multiplication by `is_source` discards the quotient at every non-SOURCE entry, and a SOURCE entry whose weight is zero lies outside every child range and so carries $\alpha_{\text{final}} = 0$. $\square$

LEMMA 4. *WINDOWCOMPUTEFOREIGNINTERVAL (Algorithm 20) can be converted to an oblivious implementation.*

PROOF. The function’s conditional logic can be made oblivious:

- (1) **Access pattern:** Always read both window slots, always write `foreign_interval` and F_{sum} of `window[1]`.

- (2) **Arithmetic conversion:** The conditional becomes:

$$\begin{aligned} \text{is_pair} &= (\text{window}[0].\text{type} == \text{START}) \\ &\quad \cdot (\text{window}[1].\text{type} == \text{END}) \in \{0, 1\} \\ \text{keep} &= 1 - \text{is_pair} \\ \text{interval} &= \text{window}[1].C_{\text{foreign}} \\ &\quad - \text{window}[0].C_{\text{foreign}} \\ \text{window}[1].\text{foreign_interval} &= \text{is_pair} \cdot \text{interval} \\ &\quad + \text{keep} \cdot \text{window}[1].\text{foreign_interval} \\ \text{window}[1].F_{\text{sum}} &= \text{is_pair} \cdot \text{window}[0].C_{\text{foreign}} \\ &\quad + \text{keep} \cdot \text{window}[1].F_{\text{sum}} \end{aligned}$$

$\square$

D.1.2 Comparators.

LEMMA 5. *COMPARATORJOINATTR (Algorithm 13) can be converted to an oblivious implementation.*

PROOF. The comparator’s conditional logic can be made oblivious:

- (1) **Access pattern:** Always read both elements’ `join_attr`, `type`, and `equality` fields, and always access the precedence table.
- (2) **Arithmetic conversion:** Convert the nested conditionals to arithmetic:

$$\begin{aligned} \text{cmp} &= \text{sign}(e_1.\text{join_attr} - e_2.\text{join_attr}) \in \{-1, 0, 1\} \\ \text{is_equal} &= (\text{cmp} == 0) \in \{0, 1\} \\ p_1 &= \text{GetPrecedence}(e_1.\text{type}, e_1.\text{equality}) \\ p_2 &= \text{GetPrecedence}(e_2.\text{type}, e_2.\text{equality}) \\ \text{prec_cmp} &= \text{sign}(p_1 - p_2) \in \{-1, 0, 1\} \\ \text{result} &= (1 - \text{is_equal}) \cdot \text{cmp} + \text{is_equal} \cdot \text{prec_cmp} \end{aligned}$$

The precedence lookup uses both type and equality type fields as indices. $\square$

LEMMA 6. *COMPARATORPAIRWISE (Algorithm 14) can be converted to an oblivious implementation.*

PROOF. The comparator has three-level comparison logic that can be made oblivious:

- (1) **Access pattern:** Always read both elements’ `type` and `orig_idx` fields.

- (2) **Arithmetic conversion:** Convert the three-level priority system:

$$\text{is_target}_1 = (e_1.\text{type} \in \{\text{START}, \text{END}\}) \in \{0, 1\}$$

$$\text{is_target}_2 = (e_2.\text{type} \in \{\text{START}, \text{END}\}) \in \{0, 1\}$$

$$\text{type_priority} = \text{is_target}_2 - \text{is_target}_1 \in \{-1, 0, 1\}$$

$$\text{idx_cmp} = \text{sign}(e_1.\text{orig_idx} - e_2.\text{orig_idx}) \in \{-1, 0, 1\}$$

$$\text{is_start}_1 = (e_1.\text{type} == \text{START}) \in \{0, 1\}$$

$$\text{is_start}_2 = (e_2.\text{type} == \text{START}) \in \{0, 1\}$$

$$\text{start_first} = \text{is_start}_2 - \text{is_start}_1 \in \{-1, 0, 1\}$$

$$\text{same_priority} = (\text{type_priority} == 0) \in \{0, 1\}$$

$$\text{same_index} = (\text{idx_cmp} == 0) \in \{0, 1\}$$

$$\text{result} = (1 - \text{same_priority}) \cdot \text{type_priority}$$

$$+ \text{same_priority} \cdot (1 - \text{same_index}) \cdot \text{idx_cmp}$$

$$+ \text{same_priority} \cdot \text{same_index} \cdot \text{start_first}$$

Priority order: (1) Target entries before SOURCE, (2) by original index, (3) START before END. $\square$

LEMMA 7. *ALIGNMENTKEYCOMPARATOR (Algorithm 16) can be converted to an oblivious implementation.*

PROOF. The comparator has three-level comparison logic that can be made oblivious:

- (1) **Access pattern:** Both elements are read in a fixed order: first alignment_key, then join_attr, then copy_index.
- (2) **Arithmetic conversion:** Convert the three-level comparison to arithmetic:

$$\text{cmp}_1 = \text{sign}(t_1.\text{alignment_key} - t_2.\text{alignment_key})$$

$$\text{eq}_1 = (t_1.\text{alignment_key} == t_2.\text{alignment_key}) \in \{0, 1\}$$

$$\text{cmp}_2 = \text{sign}(t_1.\text{join_attr} - t_2.\text{join_attr})$$

$$\text{eq}_2 = (t_1.\text{join_attr} == t_2.\text{join_attr}) \in \{0, 1\}$$

$$\text{cmp}_3 = \text{sign}(t_1.\text{copy_index} - t_2.\text{copy_index})$$

$$\text{result} = \text{cmp}_1 \cdot (1 - \text{eq}_1)$$

$$+ \text{cmp}_2 \cdot \text{eq}_1 \cdot (1 - \text{eq}_2)$$

$$+ \text{cmp}_3 \cdot \text{eq}_1 \cdot \text{eq}_2$$

This ensures deterministic ordering while maintaining obliviousness through tie-breaking on join attribute and copy index. $\square$

D.1.3 Update Functions.

LEMMA 8. *UPDATETARGETMULTIPLICITY (Algorithm 21) is inherently oblivious.*

PROOF. The function performs pure arithmetic:

- (1) **Access pattern:** Always read from both t and e , always write to $t.\alpha_{\text{local}}$.
- (2) **No conversion needed:** $t.\alpha_{\text{local}} \times e.\text{local_interval}$ is already oblivious.

$\square$

LEMMA 9. *UPDATETARGETFINALMULTIPLICITY (Algorithm 22) is inherently oblivious.*

PROOF. The function performs pure arithmetic:

- (1) **Access pattern:** Always read $e.\text{foreign_interval}$, $e.F_{\text{sum}}$, and $t.\alpha_{\text{local}}$, always write to $t.\alpha_{\text{final}}$ and $t.F_{\text{sum}}$.
- (2) **No conversion needed:** Both assignments are pure arithmetic: $t.\alpha_{\text{final}} = e.\text{foreign_interval} \times t.\alpha_{\text{local}}$ and $t.F_{\text{sum}} = e.F_{\text{sum}}$.

$\square$

D.2 Composed Operation Security

Having shown that our base components can be converted to oblivious implementations, we now prove that composing these converted oblivious versions with established oblivious primitives yields oblivious operations.

D.2.1 *Oblivious Primitives.* We rely on the following standard oblivious primitives:

ASSUMPTION 10. *The following operations are oblivious:*

- *OBLIVIOUSSORT:* as in Section A.1
- *OBLIVIOUSEXPAND:* From ODBJ [5], expands tables obliviously
- *LINEARPASS:* Iterates through a table with fixed window size 2
- *PARALLELPASS:* Applies a function to corresponding row pairs from two tables of equal size
- *MAP:* Transforms each element independently

D.2.2 Composed Operations.

LEMMA 11. *For any comparator C that can be converted to oblivious form,*

$\text{OBLIVIOUSSORT}(T, C_{\text{oblivious}})$ is oblivious.

PROOF. By Assumption 10, OBLIVIOUSSORT has a fixed comparison pattern based only on table size. By Lemmas 5-7, our comparators can be converted to oblivious implementations. Using the converted oblivious versions $C_{\text{oblivious}}$ and applying the Sequential Composition Theorem (from the main paper), the composition is oblivious. $\square$

LEMMA 12. *For any window function W that can be converted to oblivious form, $\text{LINEARPASS}(T, W_{\text{oblivious}})$ is oblivious.*

PROOF. LINEARPASS has a deterministic iteration pattern based only on table size (with fixed window size 2). By Lemmas 1-4, our window functions can be converted to oblivious implementations. Using the converted versions $W_{\text{oblivious}}$ and applying the Sequential Composition Theorem (from the main paper), the composition is oblivious. $\square$

LEMMA 13. *For any update function U that is inherently oblivious or can be converted to oblivious form, $\text{PARALLELPASS}(T, U_{\text{oblivious}})$ is oblivious.*

PROOF. PARALLELPASS applies U to each aligned pair of rows from the two tables, in index order, with a fixed access pattern. By Lemmas 8-9, our update functions are inherently oblivious (pure arithmetic). The parallel application maintains obliviousness. $\square$

D.3 Phase Security Proofs

We prove that each phase of our algorithm is oblivious.

D.3.1 Initialization Step.

LEMMA 14. *The Initialization phase (Algorithm 4) is oblivious.*

PROOF. Initialization consists of:

- (1) MAP to add metadata columns
- (2) LINEARPASS with WINDOWSETORIGINALINDEX (Alg. 6)

Both operations access each element exactly once in a predetermined order. By the Sequential Composition Theorem (from the main paper), their composition is oblivious. $\square$

D.3.2 Bottom-Up Phase.

LEMMA 15. *The Bottom-Up phase is oblivious.*

PROOF. For each node in post-order (public tree structure), the phase performs (Algorithm 9):

```
BottomUp = CombineTable (Algorithm 10)
    → ObliviousSort(ComparatorJoinAttr)
    → LinearPass(WindowComputeLocalSum)
    → ObliviousSort(ComparatorPairwise)
    → LinearPass(WindowComputeLocalInterval)
    → ObliviousSort(ComparatorEndFirst)
    → ParallelPass(UpdateTargetMultiplicity)
```

Each operation is oblivious by Lemmas 11-13. The number of iterations depends only on the public tree structure. By repeated application of the Sequential Composition Theorem (from the main paper), the entire phase is oblivious. $\square$

D.3.3 Top-Down Phase.

LEMMA 16. *The Top-Down phase is oblivious.*

PROOF. The structure mirrors the Bottom-Up phase but with pre-order traversal and different window/update functions (Algorithm 12). Each component operation is oblivious by the same arguments. By the Sequential Composition Theorem (from the main paper), the phase is oblivious. $\square$

D.3.4 Distribution and Expansion Phase.

LEMMA 17. *The Distribution and Expansion phase is oblivious.*

PROOF. This phase applies OBLIVIOUSEXPAND to each table. By Assumption 10, OBLIVIOUSEXPAND is oblivious. The operation is applied to each table independently based on the public tree structure. $\square$

D.3.5 Alignment and Concatenation Phase.

LEMMA 18. *The Alignment and Concatenation phase is oblivious.*

PROOF. For each parent-child pair, the phase performs:

- (1) OBLIVIOUSSORT on parent table with JOINTHENOTHERATTRIBUTES comparator (oblivious by Lemma 11)
- (2) LINEARPASS for alignment keys (oblivious by Lemma 12)
- (3) OBLIVIOUSSORT on the child table with ALIGNMENTKEY-COMPARATOR (Lemmas 7 and 11)
- (4) HORIZONTALCONCATENATE (oblivious concatenation)

Each operation is oblivious, and their composition is oblivious by the Sequential Composition Theorem (from the main paper). $\square$

E IMPLEMENTATION DETAILS

This section describes two parts of the implementation in full: the oblivious sorting construction, which the main paper summarizes in a sentence, and the *external-memory* (batching) configuration, a second implementation of the same algorithm that runs inside a small enclave with the data held encrypted outside it. The main paper reports no measurement from the external-memory configuration; this supplementary material is its only description and the only record of its results.

E.1 Oblivious Sorting: k-way Shuffle-then-Sort

Both configurations instantiate the shuffle-then-sort paradigm of Section A.1 with a Waksman permutation network [6] as the shuffle. They differ in what follows it.

In-enclave. The whole array is resident, so the shuffled array is sorted in place with an ordinary comparison sort over the oblivious comparators of Section B.5. There is nothing further to arrange.

External-memory: f-way shuffle and merge. Here the array does not fit, and every base case must be small enough to sort inside a 32MB enclave, so both the shuffle and the sort are made f -way recursive. An array of size n is padded to $2^a \times f^b$, where $2^a \leq \text{BATCH_SIZE}$ keeps base cases enclave-sized and $2^a \times f^b \geq n$ covers the array.

The shuffle operates recursively by first routing the input array to f subarrays using random routing, then recursively shuffling each of these f subarrays independently, and finally routing the shuffled subarrays back to the output array with random permutation. For base cases where arrays are smaller than BATCH_SIZE, we switch to a conventional 2-way Waksman network that fits entirely within the SGX enclave, ensuring efficient termination of the recursion.

The merge sort follows a bottom-up approach, starting by building sorted subarrays of size BATCH_SIZE within the SGX enclave, then recursively performing f -way merges to combine these sorted subarrays into progressively larger sorted arrays until the entire array is sorted. This f -way recursion significantly reduces the depth of the merge tree compared to binary merge sort, minimizing the number of passes over the data and the associated SGX transition overhead.

This f -way recursive approach minimizes data transfer between untrusted memory and the SGX enclave. The total amount of data transferred is $O(n \times \log_f(n/\text{BATCH_SIZE}))$, significantly reducing the overhead compared to binary recursion. We use $f = 8$ for our implementation to balance the depth of recursion with the complexity of f -way operations.

E.2 External-Memory Configuration: Batch Processing and SGX Architecture

The external-memory configuration is a hybrid architecture that combines untrusted storage with trusted SGX processing. Tables are stored encrypted under AES-CTR, each row with a unique sequential nonce so that individual rows can be fetched and re-written without decrypting anything else. The untrusted application orchestrates the algorithm's phases—sequencing the linear passes, sorts, and parallel operations—while the enclave performs the oblivious

computation itself. The enclave is sized independently of the data—32MB in most of our experiments, 128MB for the TB workload—so the dataset may be far larger than enclave capacity. Each experiment states the enclave size and batch size it used.

The unit of work crossing the boundary is an *operation*: a comparator that compares two entries for sorting (e.g., join attribute comparison, alignment key comparison), a window function that processes a sliding window of entries during a linear pass (e.g., computing multiplicities, intervals, cumulative sums), or an update function that modifies a single entry (e.g., initializing metadata, setting indices, computing derived values). The untrusted application accumulates operations into batches; when a batch reaches `MAX_BATCH_SIZE` or must be flushed, it sends the encrypted rows into the enclave, which decrypts each row with its nonce, performs the oblivious operations, re-encrypts the modified rows under fresh nonces, and returns them to untrusted storage. Batching is used chiefly for the window and update functions of the linear passes; sorting has its own construction (Section E.1). `MAX_BATCH_SIZE` is a configuration parameter—the value used is stated with each experiment (8,000 for the Higgs and BM1 measurements, 40,000 for TB)—and Section F.5 measures its effect on runtime.

Batching amortizes the cost of enclave transitions across many operations while preserving obliviousness: the access pattern visible to the untrusted host is the predetermined batch structure, which depends only on the public input sizes and query shape, never on data values or computation results.

F EVALUATION DETAILS

This section holds the query listings and the measurements that did not fit in the main body: the TPC-H band-join workloads, and the full results for the external-memory configuration described in Section E. The machine, scale factors, and metrics are those of the main paper’s Evaluation section. Enclave sizes differ by configuration: in-enclave measurements use the same 4GB heap as the main paper, while external-memory measurements use the small enclave of Section E.2, whose size is stated with each experiment.

F.1 A Defect in the Baseline’s Published Artifact

While validating the baseline we found that its published artifact returns a wrong result on some queries: one element is misrouted during oblivious distribution, so a single row that does not belong appears in the output and a single row that does belong is missing. The failure is silent, because the row count is always correct, and it is plan-dependent: it fires on both TM1 plans at scale 0.1 and on four of TM3’s fourteen plans at scale 0.01, while other plans over the same data are unaffected.

The cause is a boundary case in the distribution step, and a one-line repair fixes it. With that repair applied, the baseline returns correct results on every query in our workload that it runs. We therefore report all of its timings against the repaired build: timing a system on a knowingly incorrect result would not be a fair comparison, and the repair is in any case too small to matter, costing about 0.5% of Execution — within run-to-run variation. Speed and correctness are independent findings here, and the defect buys the baseline no performance.

We report this because it affects anyone reproducing our comparison from the upstream artifact, not as a criticism of the system: the defect is a single misrouted index, and the authors’ design is unaffected by it.

F.2 Query Listings

The full SQL for the TPC-H queries evaluated in the main paper follows.

TM1 Query:

```
SELECT * FROM customer, orders, lineitem
WHERE customer.C_CUSTKEY = orders.O_CUSTKEY
      AND lineitem.L_ORDERKEY = orders.O_ORDERKEY
```

TM2 Query:

```
SELECT * FROM supplier, customer, nation1, nation2
WHERE supplier.S_NATIONKEY = nation1.N1_N_NATIONKEY
      AND customer.C_NATIONKEY = nation2.N2_N_NATIONKEY
      AND nation1.N1_N_REGIONKEY = nation2.N2_N_REGIONKEY
```

TM3 Query:

```
SELECT * FROM nation, supplier, customer,
              orders, lineitem
WHERE nation.N_NATIONKEY = supplier.S_NATIONKEY
      AND nation.N_NATIONKEY = customer.C_NATIONKEY
      AND customer.C_CUSTKEY = orders.O_CUSTKEY
      AND orders.O_ORDERKEY = lineitem.L_ORDERKEY
```

TB1 Query:

```
SELECT * FROM supplier1, supplier2
WHERE supplier1.S1_S_ACCTBAL < supplier2.S2_S_ACCTBAL
```

TB2 Query:

```
SELECT * FROM part1, part2
WHERE part1.P1_P_RETAILPRICE < part2.P2_P_RETAILPRICE
```

BM1 Query (multi-way band join):

```
SELECT * FROM region, nation, supplier, partsupp, part
WHERE nation.N_REGIONKEY = region.R_REGIONKEY
      AND supplier.S_NATIONKEY = nation.N_NATIONKEY
      AND partsupp.PS_SUPPKEY = supplier.S_SUPPKEY
      AND partsupp.PS_SUPPLYCOST > part.P_RETAILPRICE - W
      AND partsupp.PS_SUPPLYCOST < part.P_RETAILPRICE
```

F.3 TPC-H Workloads: Band Joins and External-Memory Results

This subsection is the primary record for the TPC-H band-join workloads, since no measured baseline supports band predicates: the binary band joins **TB1** (account balances between two supplier tables) and **TB2** (retail prices between two part tables) from the OJOIN query set, and **BM1**, our 5-way band join over region, nation, supplier, partsupp, part, which finds every (part, supplier) pair where the supplier’s cost is within W price units below the part’s retail price. Times for the external-memory configuration of Section E are Totals; its small enclave initializes in a fraction of a second, so $\text{Total} \approx \text{Execution}$. The BM1 and Higgs tables set these against the in-enclave Execution times of the main paper. The TPC-H table does not: TB1 and TB2 have no in-enclave Execution measurement, and the main paper’s TM table already carries that comparison for the TM series. Both configurations execute

the same algorithm. Every in-enclave result is verified tuple-exact against a plaintext reference; the external-memory results are likewise verified tuple-exact, except for TM3 at scale 0.1, where only the row count has been checked. The external-memory configuration’s per-batch ECALL/OCALL cost makes its join time slower throughout—by roughly an order of magnitude on mid-sized workloads, narrowing to 1.2 $\times$ on the heaviest Higgs cell—which is why the main body reports in-enclave results. Unless noted otherwise the batch size is 8,000 operations with a 32MB enclave (TB uses 40,000 with a 128MB enclave). In BM1, in-enclave runtime tracks the output size: wider bands admit more matches, and runtime grows with the output they select—consistent with the output-sensitive bound demonstrated on Higgs in the main paper.

Table 1: External-memory Total (seconds) on the TPC-H workloads. “–”: could not complete.

Query	Scale	Output	Ext.-mem. Total
TM1	0.001	6,005	4.6
TM1	0.01	60,175	33.4
TM1	0.1	600,572	254
TM2	0.001	292	0.6
TM2	0.01	29,929	8.6
TM2	0.1	2,999,594	1,342
TM3	0.001	2,485	4.4
TM3	0.01	236,250	102
TM3	0.1	24,029,033	10,151
TB1	0.001	45	0.36
TB1	0.01	4,950	1.5
TB1	0.1	499,499	80
TB2	0.001	19,900	4.4
TB2	0.01	1,998,097	337
TB2	0.1	199,915,491	–

Table 2: External-memory Total (seconds) on BM1 (5-way band join) as band width W varies, alongside in-enclave Execution.

Scale	W	Output	In-encl. Exec	Ext.-mem. Total
0.01	100	1,532	0.86	10.7
0.01	400	6,266	0.90	13.3
0.01	1,000	16,095	0.92	17.3
0.01	10,000	229,887	5.03	95.2
0.1	100	144,156	10.56	118
0.1	400	590,114	32.71	249
0.1	1,000	1,525,033	73.39	548

F.4 Higgs Workloads: External-Memory Results

The main paper reports the Higgs day ladder for the in-enclave configuration; the table below adds the external-memory configuration on the same queries. The gap narrows as the output grows—up to roughly 27 $\times$ at the 1-day rung, where the join work is sub-second and the per-batch transitions dominate, falling to single digits at the largest outputs, where oblivious compute dominates both configurations.

Table 3: Higgs day ladder ($W=4s$, batch 8,000), in seconds: in-enclave Execution against external-memory Total.

Query	k	In-encl. Exec			Ext.-mem. Total		
		1d	2d	3d	1d	2d	3d
HQ3	3	0.48	1.24	3.21	3.67	15.8	34.5
HQ4	4	0.52	5.11	55.61	9.77	126.7	432.5
HQ5	5	0.51	6.64	74.49	11.2	154.4	520.7
Chain	2	0.50	0.93	1.76	1.94	8.37	19.7
	5	0.51	1.20	2.42	7.13	29.4	67.1
	9	0.54	1.48	3.44	13.4	57.2	143.8
Tree	5	0.52	1.27	3.42	7.27	41.0	93.5
	7	0.54	3.58	46.07	11.0	96.7	380.9
	8	0.55	9.37	135.43	12.6	161.1	786.5
	9	0.55	26.58	1,992.00	14.7	259.2	2,344.55

F.5 Batch-Size Sensitivity

Batch processing trades enclave size against ECALL boundary cost: very small batches pay one ECALL per operation, while very large batches need a slow-to-initialize enclave and overflow CPU caches during the Waksman shuffle. Figure 1 confirms this on TM3 at SF 0.01: total runtime is U-shaped in batch size. The sweep was measured on an earlier build, before the AVX2 and split-row optimizations, so its absolute times run above those reported for the same cell elsewhere in this section; what it establishes is the shape of the curve and the location of the optimum, both of which are properties of the batching architecture rather than of a particular build. Runtime falls steeply as batching amortizes ECALL transitions (340 s at batch size 8 down to 134 s at 2,000), then rises again as the shuffle working set overflows the L2 and L3 caches (visible as steps near batch sizes 8,000 and 40,000). As part of this experiment we also decompose the optimum configuration’s runtime into in-enclave computation (104 s: cryptography and oblivious operations) and out-of-enclave orchestration and I/O (30 s), quantifying where time is spent.

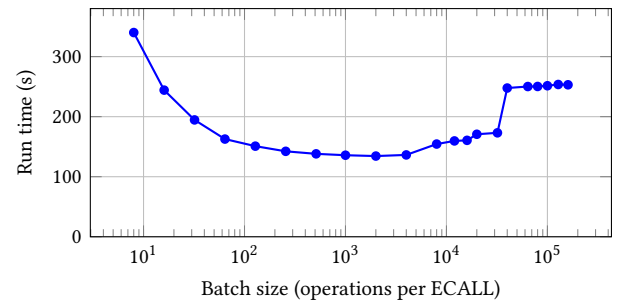


Figure 1: Batch-size sweep on TM3 (5-way equality join, TPC-H SF 0.01). Runtime is U-shaped: small batches are dominated by ECALL transitions, large batches by shuffle cache overflow.

REFERENCES

- [1] K. E. Batchier. 1968. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference* (Atlantic City, New

- Jersey) (*AFIPS '68 (Spring)*). Association for Computing Machinery, New York, NY, USA, 307–314. <https://doi.org/10.1145/1468075.1468121>
- [2] Michael T. Goodrich. 2014. Zig-zag sort: a simple deterministic data-oblivious sorting algorithm running in $O(n \log n)$ time. In *Proceedings of the Forty-Sixth Annual ACM Symposium on Theory of Computing* (New York, New York) (*STOC '14*). Association for Computing Machinery, New York, NY, USA, 684–693. <https://doi.org/10.1145/2591796.2591830>
 - [3] Georg Gottlob, Nicola Leone, and Francesco Scarcello. 2002. Hypertree Decompositions and Tractable Queries. *J. Comput. Syst. Sci.* 64, 3 (May 2002), 579–627. <https://doi.org/10.1006/jcss.2001.1809>
 - [4] Koki Hamada, Ryo Kikuchi, Dai Ikarashi, Koji Chida, and Katsumi Takahashi. 2013. Practically Efficient Multi-party Sorting Protocols from Comparison Sort Algorithms. In *Information Security and Cryptology – ICISC 2012 (Lecture Notes in Computer Science)*, Vol. 7839. Springer, 202–216.
 - [5] Simeon Krastnikov, Florian Kerschbaum, and Douglas Stebila. 2020. Efficient oblivious database joins. *Proc. VLDB Endow.* 13, 12 (July 2020), 2132–2145. <https://doi.org/10.14778/3407790.3407814>
 - [6] Abraham Waksman. 1968. A Permutation Network. *J. ACM* 15, 1 (Jan. 1968), 159–163. <https://doi.org/10.1145/321439.321449>
 - [7] Mihalis Yannakakis. 1981. Algorithms for acyclic database schemes. In *Proceedings of the Seventh International Conference on Very Large Data Bases - Volume 7* (Cannes, France) (*VLDB '81*). VLDB Endowment, Cannes, France, 82–94.